CÓMPUTO PARALELO Y EN LA NUBE

DBSCAN paralelo para detección de ruido

Descripción técnica y evaluación experimental individual

Proyecto: victormbr-creator/dbscan-openmp
Fecha de preparación: 15 de octubre de 2026
Entrega indicada: 20 de octubre de 2026

Alcance	Evidencia
Implementación	C++17: P0 serial, P1 OpenMP estático y P2 OpenMP por bloques dinámicos.
Evaluación	116 configuraciones, 10 repeticiones y 1160 mediciones válidas.
Resultado destacado	Speedup de 6.24 con P2, 200000 puntos, 10 hilos y 64 particiones.
Corrección	Ocho casos frente a scikit-learn y equivalencia contra P0 de las 116 configuraciones.

Se clasifican puntos core de primer orden, segundo orden (borde) y ruido. No se asignan identificadores de clúster. El presente documento conserva los resultados medidos; no presenta tiempos estimados como mediciones.

Repositorio privado: github.com/victormbr-creator/dbscan-openmp. Los CSV de respaldo están en results/benchmarks/e5129e76b440779f/.

1. Algoritmo y datos

El vecindario de un punto contiene los puntos a distancia menor o igual que epsilon, incluido el propio punto. Un punto es core si su vecindario contiene al menos min_samples observaciones. Un no-core es de segundo orden si tiene algún core vecino; de lo contrario es ruido.

Fase 1: recorrer los posibles vecinos de cada punto y marcar core al alcanzar el umbral. Fase 2: para cada no-core, buscar un core vecino. Los bordes no propagan la condición de core. Las clases de salida son 0=ruido, 1=core y 2=segundo orden.

Se compara $dx^2 + dy^2 <= epsilon^2$, sin raíz cuadrada. La búsqueda se detiene cuando alcanza min_samples o encuentra el primer core vecino, según la fase. Se calcula la condición de umbral, no necesariamente el conteo total. Estas optimizaciones son comunes a P0, P1 y P2.

La memoria principal es O(N); el peor caso temporal sigue siendo $O(N^2)$. No se construye una matriz de distancias. La parada temprana reduce el trabajo en los datos densos estudiados, pero no garantiza esa reducción en otros conjuntos.

Generación reproducible

Parámetro	Valor
Generador	sklearn.datasets.make_blobs
centers / cluster_std	4 / 0.06
random_state / center_box	11 / (0, 1)
Transformación	Inversión de columnas, valor absoluto y redondeo a 3 decimales.
Clasificación	epsilon=0.03; min_samples=10
Formato	CSV sin encabezados, dos columnas numéricas x,y.

scripts/generar_datos.py determina la ruta data/ desde la ubicación del script, independientemente del directorio de ejecución. Verifica forma, valores finitos e igualdad exacta al recargar. Esto resuelve la dependencia de la ruta relativa del notebook.

Estructura: src/dbscan_common.hpp comparte lectura y reglas; serial.cpp, paralelo1.cpp y paralelo2.cpp seleccionan la estrategia. Los scripts generan, validan, miden, auditan y grafican.

2. Paralelización y alcance de P2

Versión	Distribución	Consulta de vecinos
P0	Bucles secuenciales	Todo el conjunto
P1	omp for schedule(static), sobre puntos	Todo el conjunto
P2	omp for schedule(dynamic,1), sobre k bloques de índices	Todo el conjunto

Para cada bloque b, P2 procesa los índices desde floor(N*b/k) hasta floor(N*(b+1)/k), excluyendo el extremo final. Los intervalos cubren todos los puntos sin superposición. No se crean copias de los datos ni particiones espaciales aisladas.

Cada hilo escribe sólo los elementos correspondientes a sus puntos objetivo. Los datos se mantienen de sólo lectura. La barrera implícita al terminar la fase 1 completa el vector global core antes de comenzar la fase 2. Se utilizan bytes independientes y se evita vector<bool>.

Tratamiento de resultados entre particiones

La partición reparte el trabajo, no limita el vecindario. Un punto del bloque A puede contar vecinos y consultar cores del bloque B. Las contribuciones de cada bloque quedan directamente en posiciones disjuntas de los vectores compartidos; la barrera hace disponible el estado global completo. No hay una fase independiente de fusión de clasificaciones locales ni reconciliación de clústeres.

Según el contenido de la diapositiva 11 proporcionado por el estudiante, P2 debe dividir la matriz en k partes, procesarlas en paralelo y mantener consistencia al unir sus resultados, sin exigir un algoritmo concreto de reconciliación. La partición lógica de filas y composición directa por índices exclusivos responde técnicamente a ese planteamiento; las consultas globales preservan los vecinos entre bloques. Esta justificación no equivale a una aprobación expresa del profesor. La barrera sincroniza las fases y no realiza una fusión de clasificaciones locales.

Si se exigiera una unión explícita, una alternativa sería contar vecinos locales por pares de bloques y sumar las contribuciones antes de decidir cores; después combinar mediante OR la existencia de cores vecinos en cada bloque. Esa variante cambia el trabajo y requiere nueva validación y medición. No forma parte de los resultados aquí reportados.

P1 reduce coordinación mediante reparto estático. P2 ofrece más oportunidades de balance cuando hay suficientes bloques; si k es menor que el número de hilos, sólo k hilos pueden procesar bloques simultáneamente en una fase. No se instrumentó el tiempo por hilo para atribuir causalidad definitiva a las diferencias de rendimiento.

3. Entorno y diseño experimental

Elemento	Configuración registrada
Equipo	Apple M5; 10 procesadores físicos y 10 lógicos; 24 GiB RAM.
Sistema	macOS 26.5.2, build 25F84, arm64.
Compilador	Homebrew clang 23.1.3; OpenMP para P1/P2.
Opciones comunes	-std=c++17 -O3 -Wall -Wextra -Wpedantic -ffp-contract=off
Paquetes Python	NumPy 2.5.4; scikit-learn 1.9.1; Matplotlib 3.11.2.
N individual	20000, 80000, 140000, 200000.
Hilos T	1, 5, 10, 20.
Particiones k	2, 4, 8, 16, 32, 64.

El perfil individual selecciona cuatro de los ocho tamaños solicitados y conserva todas las configuraciones de hilos y particiones. Por tamaño: 1 configuración P0 + 4 P1 + 24 P2 = 29. Cuatro tamaños dan 116 configuraciones; diez repeticiones dan 1160 mediciones. Los 20 hilos implican sobresuscripción sobre 10 procesadores lógicos.

steady_clock mide la inicialización de etiquetas, ambas fases, entrada al equipo OpenMP y sincronización. Excluye lectura, conteo final de categorías, escritura CSV y salida JSON. Se utiliza el mismo dataset para comparar las tres versiones.

Se descarta un calentamiento por configuración y se intercalan las diez repeticiones en orden pseudoaleatorio de semilla 11. Se conservan tiempos individuales, medias y desviaciones estándar, sin eliminar observaciones. La carga y la temperatura pueden afectar los tiempos.

Speedup = media(P0) / media(paralelo). Eficiencia = speedup / hilos. La eficiencia no equivale al porcentaje de uso de CPU. Variar hilos con N fijo evalúa escalabilidad fuerte; variar N no constituye por sí solo una prueba formal de escalabilidad débil.

4. Resultados y variabilidad

N	Versión	T / k	Media ms	DE ms	Speedup
20000	P0	1 / -	9.4525	0.1542	1.000
20000	P1	10 / -	1.9046	0.0743	4.963
20000	P2	10 / 64	1.6737	0.0798	5.648
80000	P0	1 / -	40.5661	0.6320	1.000
80000	P1	20 / -	7.5737	0.1274	5.356
80000	P2	10 / 64	6.5907	0.2365	6.155
140000	P0	1 / -	74.3209	1.9423	1.000
140000	P1	20 / -	13.5729	0.1850	5.476
140000	P2	10 / 64	11.9734	0.2083	6.207
200000	P0	1 / -	104.3382	3.1925	1.000
200000	P1	20 / -	19.0317	0.4159	5.482
200000	P2	10 / 64	16.7330	0.3900	6.235

Tabla 1. Mejores medias observadas por versión y tamaño. Cada fila resume diez repeticiones. DE es desviación estándar muestral, no intervalo de confianza. La selección de la mejor configuración es descriptiva; todas las configuraciones están disponibles en summary.csv.

Ambas versiones paralelas superaron 1.5 en los cuatro tamaños evaluados. P2 con 10 hilos y 64 bloques obtuvo la mejor media en todos. Para 200000 puntos, la mejor P2 redujo el tiempo aproximadamente 12.1% frente a la mejor P1. Su eficiencia fue 62.35%; la de P1 con 20 hilos fue 27.41%.

No se garantiza que la mejor configuración se mantenga en otros equipos, distribuciones o parámetros. En estos datos, la alta densidad permite parada temprana; el crecimiento aproximadamente lineal observado no cambia la complejidad de peor caso.

5. Tiempo y escalabilidad

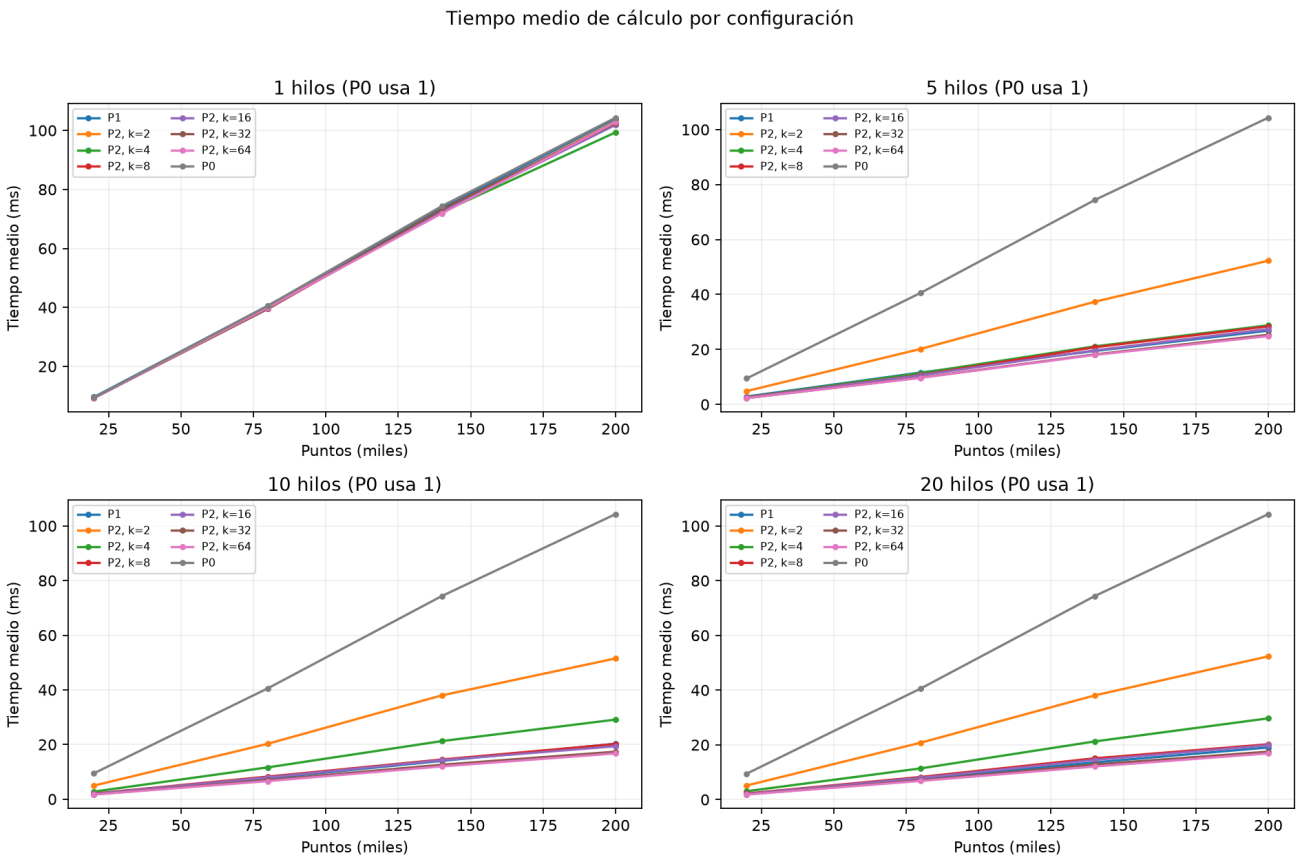


Figura 1. Tiempo medio de cálculo en milisegundos. Cada panel fija el número de hilos de las versiones paralelas; P0 siempre usa ejecución serial. Se muestran todos los k de P2.

El tiempo aumenta con N. En P2, k=2 limita el trabajo concurrente a dos bloques y deja de aprovechar hilos adicionales. Con más bloques se observan menores tiempos. Las curvas corresponden a la clasificación y no incluyen entrada/salida.

El estudio conserva comparabilidad algorítmica: no se usa un índice espacial exclusivo de una versión, no hay matrices precalculadas y las condiciones de parada son compartidas.

6. Speedup frente a hilos

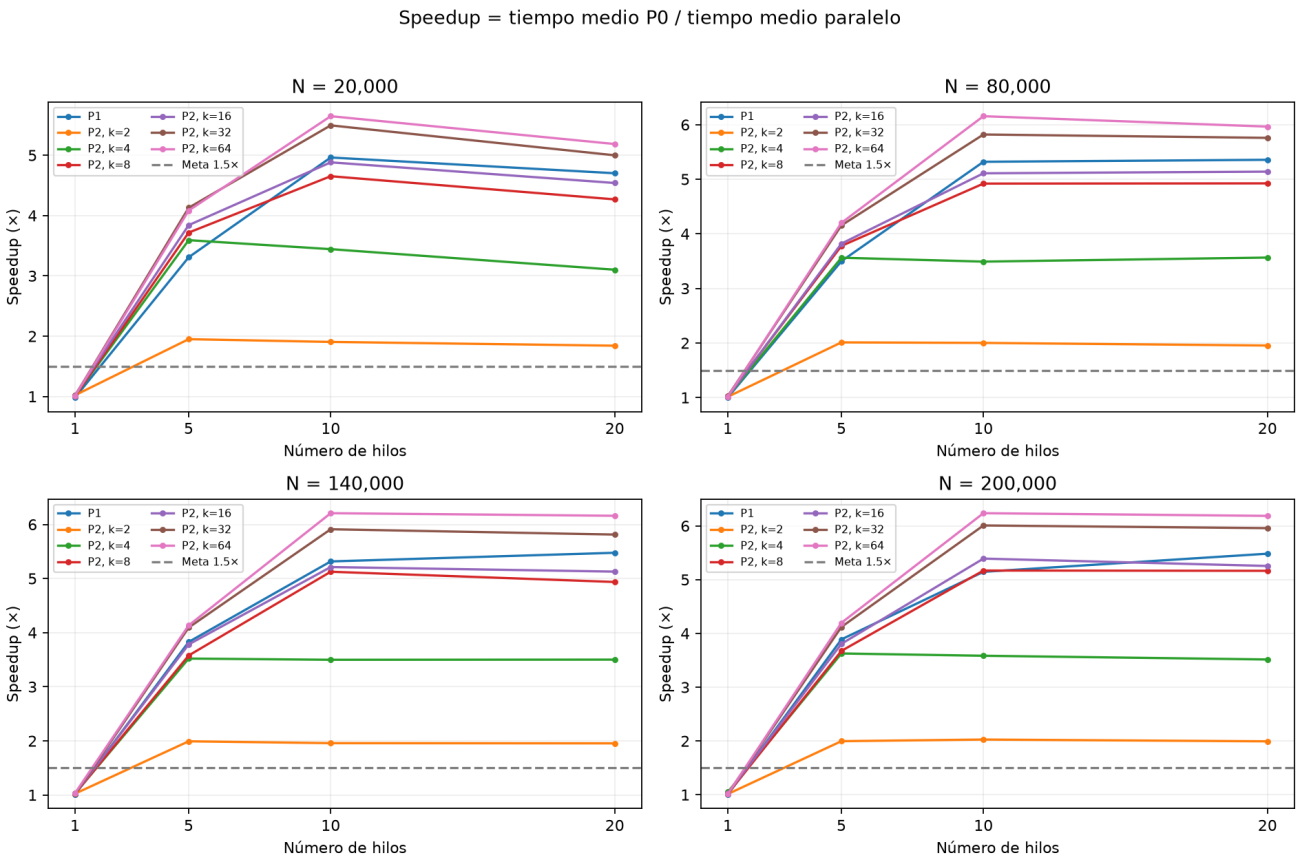


Figura 2. Speedup respecto a la media serial del mismo N. La línea discontinua marca el objetivo 1.5. Cada panel corresponde a un tamaño; las curvas de P2 conservan k fijo.

P2 alcanza su mejor media con 10 hilos y k=64. El paso a 20 hilos no mejora esa configuración. P1 alcanza mejores medias con 20 hilos en los tres tamaños mayores, pero el efecto no debe generalizarse: no se midieron directamente planificación, temperatura ni desequilibrio por hilo.

7. Speedup frente a particiones

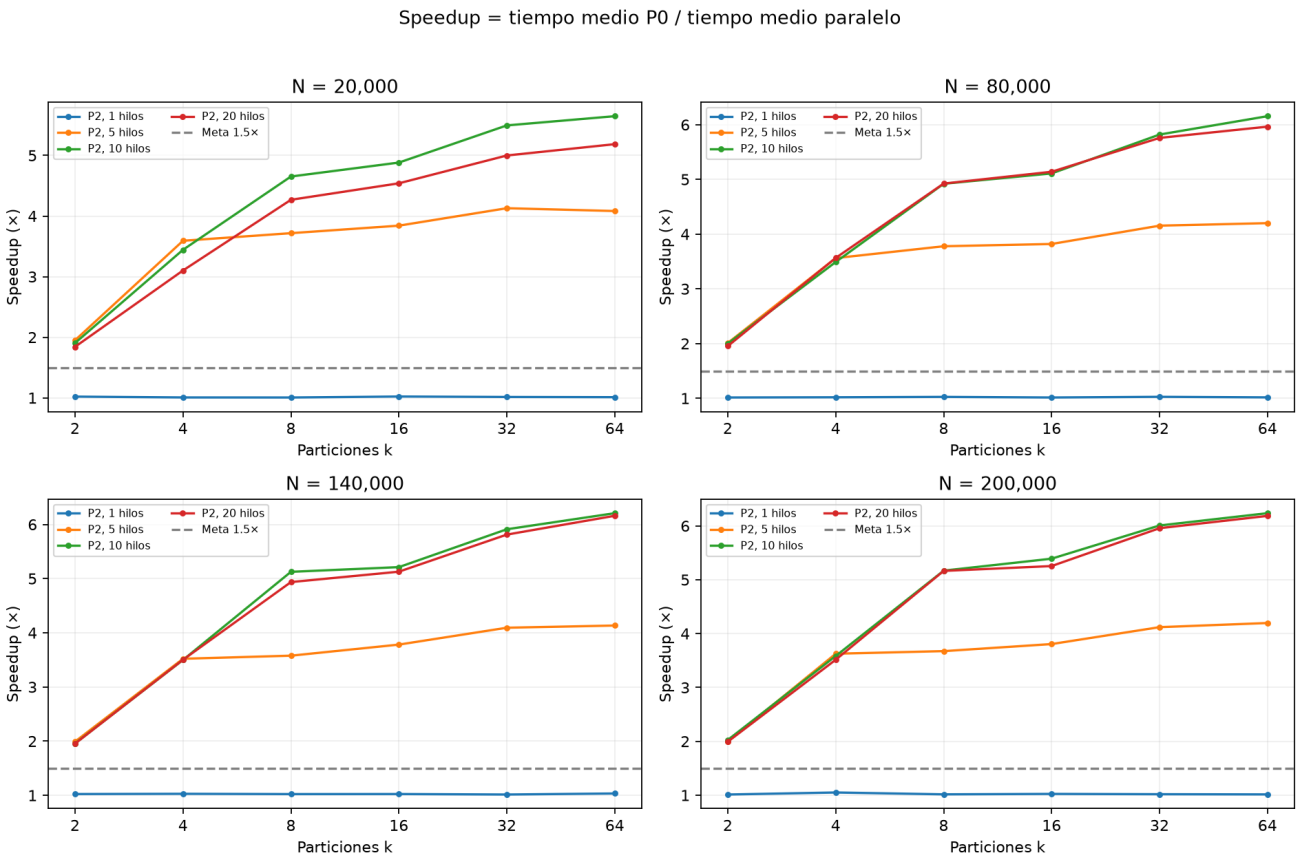


Figura 3. Speedup de P2 al variar k; el eje horizontal muestra potencias de dos equiespaciadas. Cada curva fija el número de hilos.

Con un hilo el speedup permanece cerca de uno. Con varios hilos, k=2 mantiene una aceleración cercana a dos. Aumentar k hasta 64 mejora el rendimiento medido con 10 hilos. Esto es compatible con mejor balance de carga, aunque no demuestra por sí solo su causa.

8. Validación, reproducción y uso de IA

La referencia externa usa DBSCAN de scikit-learn con algorithm=kd_tree, core_sample_indices_ para cores y labels_=-1 para ruido. Los no-core restantes son borde. Se probaron datos del profesor, duplicados, distancia epsilon exacta, aislados, densos, un punto, min_samples mayor que N y vecinos entre bloques: 120 ejecuciones en total.

La primera comparación con el backend brute mostró cinco discrepancias en 4000 puntos; kd_tree coincidió con la implementación. Se documenta sensibilidad al cálculo flotante cerca del umbral, sin tolerancias arbitrarias ni garantía de identidad entre todos los backends.

La auditoría de la matriz revisó hashes, diez repeticiones distintas por configuración, tiempos finitos y positivos, medias, speedup y eficiencia. Reejecutó las 116 configuraciones y comparó los CSV completos contra P0. Los datasets grandes se contrastaron con P0; no se afirma validación externa de cada uno con sklearn.

N	Core	Segundo orden	Ruido
20000	19866	89	45
80000	79885	81	34
140000	139896	69	35
200000	199902	77	21

Reproducción limpia

Se clonó el repositorio remoto en una carpeta temporal, se creó un entorno virtual nuevo, se instalaron dependencias, se compilaron los ejecutables y se generaron los datos sin reutilizar binarios ni datasets locales. El detalle de comandos, versión y resultados está en docs/reproduccion_limpia.md. La prueba usa el mismo Mac y las dependencias de sistema instaladas; no demuestra portabilidad a otro sistema operativo.

Se utilizó Codex de OpenAI para asistir en código, automatización, pruebas y documentación. Los resultados proceden de ejecuciones reales. La modalidad individual fue confirmada por el estudiante; la revisión y comprensión para la defensa siguen siendo su responsabilidad.

Archivos de respaldo: docs/metodologia.md, docs/resultados.md, docs/uso_ia.md y results/benchmarks/e5129e76b440779f/{raw.csv,summary.csv,metadata.json,validation.csv}. El PDF fue generado por scripts/generar_informe.py a partir de esos resultados.